[28/10, 17:14] Unnati Chavan🌸: # Practical 6: Execute a Jenkins Pipeline

**Aim:** Create a Jenkins pipeline that prints `Welcome to DevOps Practical` and `Build Successful` in the console output.

## Part A: Install and Start Jenkins Using Docker

### Step 1: Open PowerShell
Open VS Code terminal or PowerShell.

### Step 2: Download the Jenkins image

```powershell
docker pull jenkins/jenkins:lts
```

Wait until the image download completes.

### Step 3: Create and start the Jenkins container

```powershell
docker run -d --name jenkins-practical -p 8082:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts
```

### Step 4: Verify the container

```powershell
docker ps
```

Check that `jenkins-practical` has status `Up`.

### Step 5: Open Jenkins
Open Chrome and visit:

`http://localhost:8082`

Wait for Jenkins to finish starting.

### Step 6: Unlock Jenkins
When the **Unlock Jenkins** page appears, run this command in PowerShell:

```powershell
```

docker exec jenkins-practical cat /var/jenkins_home/secrets/initialAdminPassword
```

Copy the password printed in the terminal and paste it into the Jenkins webpage.

### Step 7: Install plugins
1. Click **Install suggested plugins**.
2. Wait for installation to finish.
3. Create an administrator account if prompted.
4. Continue to the Jenkins dashboard.

## Part B: Create the Pipeline

### Step 8: Create a new item
1. Click **New Item**.
2. Enter the name `DevOpsPipeline`.
3. Select **Pipeline**.
4. Click **OK**.

### Step 9: Enter the pipeline script
Scroll to the **Pipeline** section.

Set **Definition** to `Pipeline script`.

Paste the following code:

```groovy
pipeline {
    agent any

    stages {
        stage('Welcome') {
            steps {
                echo 'Welcome to DevOps Practical'
                echo 'Build Successful'
            }
        }
    }
}
```

Click **Save**.

## Part C: Execute and Verify
```

### Step 10: Run the pipeline
1. Open `DevOpsPipeline`.
2. Click **Build Now**.
3. Wait for the build to finish.

### Step 11: Check console output
1. Click the build number under **Build History**.
2. Click **Console Output**.

**Expected output:**

```text
Welcome to DevOps Practical
Build Successful
Finished: SUCCESS
```

**Result:** The Jenkins pipeline executed successfully and printed both required messages.

## Important commands for revision

Check running containers:

```powershell
docker ps
```

Start Jenkins again if its container is stopped:

```powershell
docker start jenkins-practical
```

Open Jenkins:

`http://localhost:8082`

**Note:** The first-time installation steps are normally required only once. For subsequent practical sessions, start the existing container if necessary and open Jenkins. Keep the `jenkins-practical` container and `jenkins_home` volume so your configuration and pipeline are preserved.
[28/10, 17:15] Unnati Chavan🌸: # Practical 9: Kubernetes — Nginx with 3 Replicas

## Step 1: Check Kubernetes
Open Docker Desktop and check whether Kubernetes is enabled.

Open VS Code terminal and run:

```powershell
kubectl get nodes
```

## Step 2: Create the deployment file
Create a folder named `KubernetesPractical` inside `C:\Users\admin\OneDrive\Documents`.

Inside it, create a file named `deployment.yaml` and write:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:alpine
        ports:
        - containerPort: 80
```

Save with Ctrl + S.

## Step 3: Create the deployment
In the VS Code terminal, from inside the `KubernetesPractical` folder, run:

```powershell
kubectl apply -f deployment.yaml
```

## Step 4: Verify the deployment and pods

```powershell
kubectl get deployments
kubectl get pods
```

**Expected result:** Deployment shows 3 desired and 3 ready replicas, and 3 Nginx pods eventually show `Running`.

## Step 5: Clean up after the practical (optional)

```powershell
kubectl delete -f deployment.yaml
```

**Result:** An Nginx Kubernetes Deployment runs with three replicas.
[28/10, 17:15] Unnati Chavan🌸: # Practical 8: Docker Compose — Nginx Web Server

## Step 1: Create a folder
Open VS Code → Open Folder → create and open:

`C:\Users\admin\DockerComposePractical`

## Step 2: Create the Compose file
Create a file named exactly `compose.yaml` inside that folder.

Write this code:

```yaml
services:
  web:
    image: nginx:alpine
    ports:
      - "8081:80"
```

Save the file using Ctrl + S.

## Step 3: Start the web server
Open the VS Code terminal and run:

```powershell
```

docker compose up -d
```

## Step 4: Verify
Run:

```powershell
docker compose ps
```

Open Chrome and visit:

`http://localhost:8081`

The default Nginx welcome page should appear.

## Step 5: Stop the service
When you want to stop it, run:

```powershell
docker compose down
```

**Result:** The Nginx web server runs successfully using Docker Compose.
[28/10, 17:17] Unnati Chavan🌸: # Practical 7: Build a Docker Image

**Aim:** To create a Docker image for a simple HTML page displaying “Welcome to Docker”, build the image, and run it in a container.

### Step 1: Open VS Code
1. Open VS Code.
2. Select Terminal → New Terminal.
3. Create a project folder:

```powershell
cd C:\Users\admin
mkdir DockerPractical
cd DockerPractical
```

If the folder already exists, use `cd C:\Users\admin\DockerPractical`.

### Step 2: Create the HTML file
Inside `C:\Users\admin\DockerPractical`, create `index.html` and enter:
```

```html
<!DOCTYPE html>
<html>
<head>
   <title>Docker Practical</title>
</head>
<body>
   <h1>Welcome to Docker</h1>
</body>
</html>
```

Save the file.

### Step 3: Create the Dockerfile
In the same folder, create a file named exactly `Dockerfile` (no extension) and enter:

```dockerfile
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
```

Save it.

### Step 4: Build the Docker image

```powershell
docker build -t welcome-docker .
```

### Step 5: Run the container

```powershell
docker run -d -p 8080:80 --name docker-web welcome-docker
```

### Step 6: Verify
Open a browser and visit:

http://localhost:8080

The page should display **Welcome to Docker**.

### Useful commands

```powershell
docker images
docker ps
docker stop docker-web
```

**Expected result:** The Docker image is built successfully and the HTML page runs in a container.

**Result:** Successfully built and executed a Docker image.
[28/10, 22:22] Unnati Chavan🌸: # Practical 12 — Grafana Dashboard

## Aim
To create a Grafana dashboard with a panel displaying CPU or memory usage from an available metrics source.

## Software Requirements
- Docker Desktop
- Docker Compose
- Grafana
- Prometheus

## Theory
- **Grafana:** Visualizes metrics using dashboards and graphs.
- **Prometheus:** Collects and stores time-series metrics.
- **Dashboard:** Collection of visualization panels.
- **Panel:** Displays a graph, gauge, or value.
- **Data source:** Provides data to Grafana.

## Step 1: Create the project folder

Create this folder inside Documents:

`GrafanaPractical`

Open the folder, click the File Explorer address bar, type `cmd`, and press Enter.

## Step 2: Create `compose.yaml`

Create a file named `compose.yaml` inside `GrafanaPractical`.

Add:

```yaml
services:
  prometheus:
    image: prom/prometheus
    ports:
      - "9092:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml

  grafana:
    image: grafana/grafana
    ports:
      - "3001:3000"
    depends_on:
      - prometheus
```

The host ports are `9092` for Prometheus and `3001` for Grafana because the usual ports were already occupied on this computer.

## Step 3: Create `prometheus.yml`

Create another file in the same folder named `prometheus.yml`.

Add:

```yaml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ["prometheus:9090"]
```

Save both files.

## Step 4: Start the containers

Run:

```powershell
```

docker compose up -d
```

Check the status:

```powershell
docker compose ps
```

Both services should show as running.

## Step 5: Open the applications

- Grafana: `http://localhost:3001`
- Prometheus: `http://localhost:9092`

Grafana login for a fresh instance:
- Username: `admin`
- Password: `admin`

Change the password if prompted.

## Step 6: Add Prometheus as a Grafana data source

1. In Grafana, open **Connections → Data sources**.
2. Click **Add data source**.
3. Select **Prometheus**.
4. Enter the server URL:

   `http://host.docker.internal:9092`

5. Click **Save & test**.

The connection should succeed.

**Note:** This host URL is the working connection for this setup. If the two containers are configured on the same Compose network, the internal URL `http://prometheus:9090` can also be used.

## Step 7: Create a dashboard panel

1. Open **Dashboards → New dashboard**.
2. Click **Add visualization** or use the panel editor.
3. Select the Prometheus data source.
```

4. Switch the query editor to **Code**.
5. Enter:

```promql
up
```

6. Click **Run queries**.
7. Choose **Stat** or **Time series** visualization.
8. Click **Apply**.
9. Save the dashboard as `Grafana Dashboard`.

The query `up` displays target availability: `1` means the target is reachable; `0` means it is not.

## Step 8: CPU or memory metrics

The basic configuration above verifies that Prometheus and Grafana work together, but it does not collect actual computer CPU or memory usage.

To display real CPU or memory usage, configure a suitable exporter, such as **cAdvisor** for container resource metrics, or a compatible system metrics exporter. Add it as a Prometheus scrape target, then use the corresponding metric in the Grafana panel.

## Useful Docker Commands

| Command | Purpose |
|---|---|
| `docker --version` | Checks Docker installation |
| `docker compose version` | Checks Compose installation |
| `docker compose up -d` | Starts the services |
| `docker compose ps` | Checks service status |
| `docker compose logs` | Displays service logs |
| `docker compose down` | Stops and removes the Compose containers and network |

## Expected Output
- Grafana opens successfully.
- Prometheus is connected as a data source.
- A dashboard panel displays the `up` metric.
- With a suitable exporter configured, the dashboard can display CPU or memory usage.

## Result
A Grafana dashboard was created and connected to Prometheus to visualize available monitoring metrics.

## Viva Questions and Answers

**1. What is Grafana?**
A tool used to visualize and monitor data.

**2. What is Prometheus?**
A monitoring system that collects and stores time-series metrics.

**3. What is a dashboard?**
A collection of panels displaying information.

**4. What is a panel?**
An individual graph, gauge, table, or statistic.

**5. What is a data source?**
A source from which Grafana retrieves data.

**6. What is PromQL?**
The query language used by Prometheus.

**7. What does `up` mean?**
It shows whether a monitored target is reachable.

**8. What is Docker Compose?**
A tool for defining and running multiple containers using YAML.

**9. Why is Prometheus used with Grafana?**
Prometheus collects metrics, while Grafana visualizes them.

**10. Does Grafana itself collect CPU metrics?**
No. It visualizes metrics provided by a data source.

**11. What does `docker compose up -d` do?**
It starts services in the background.

**12. What is an exporter?**
A component that exposes system or application metrics for Prometheus to collect.
[28/10, 22:54] Unnati Chavan🌸: # Practical 11: Selenium — Automate Google Search

**Aim:** Open Google using Selenium, search for `DevOps`, and print the page title.

## Part A: Setup

### Step 1: Create a folder

Open VS Code → File → Open Folder.

Create and open:

`C:\Users\admin\OneDrive\Documents\SeleniumPractical`

### Step 2: Check Python

Open the VS Code terminal and run:

```powershell
python --version
```

### Step 3: Install Selenium

```powershell
python -m pip install selenium
```

Wait for the installation to finish.

## Part B: Write the program

### Step 4: Create a Python file
Inside the `SeleniumPractical` folder, create a file named `selenium_practical.py`.

### Step 5: Enter this code

```python
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait

driver = webdriver.Chrome()

driver.get("https://www.google.com")

search_box = driver.find_element(By.NAME, "q")
search_box.send_keys("DevOps")
search_box.send_keys(Keys.RETURN)

WebDriverWait(driver, 15).until(
```

```
    lambda d: "DevOps" in d.title
)

print("Page Title:", driver.title)

driver.quit()
```

Save the file using Ctrl + S.

## Part C: Execute and verify

### Step 6: Run the script

```powershell
python selenium_practical.py
```

Chrome should open, search for `DevOps`, and display the results.

### Step 7: Check the output

The terminal should print a title similar to:

`Page Title: DevOps - Google Search`

The exact title may vary.

**Result:** Selenium successfully searched Google for DevOps and printed the page title.

**Note:** Selenium 4 can usually manage the Chrome driver automatically. Keep Chrome installed and internet access enabled.